



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

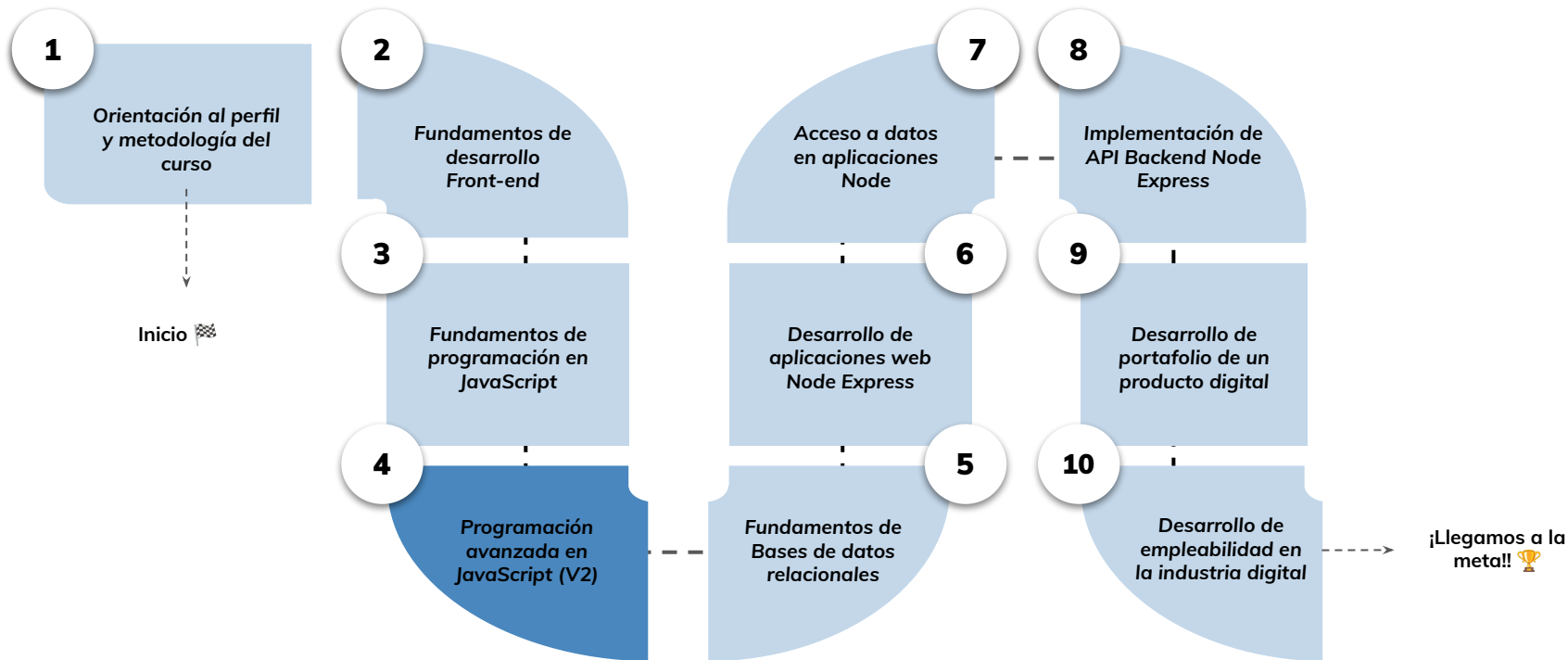


› Eventos y manipulación del DOM -Parte II

AE3.2: Reconocer los elementos fundamentales del Domain Object Model y los mecanismos para la manipulación de elementos en un documento html.

Hoja de ruta

¿Cuáles skills conforman el programa?



Hoja de ruta

¿Cuáles skills conforman el programa?

Orientación a objetos
en JavaScript

1

Eventos y manipulación
del DOM

3

Consumo de apis con
JavaScript

5

Características JavaScript
es6

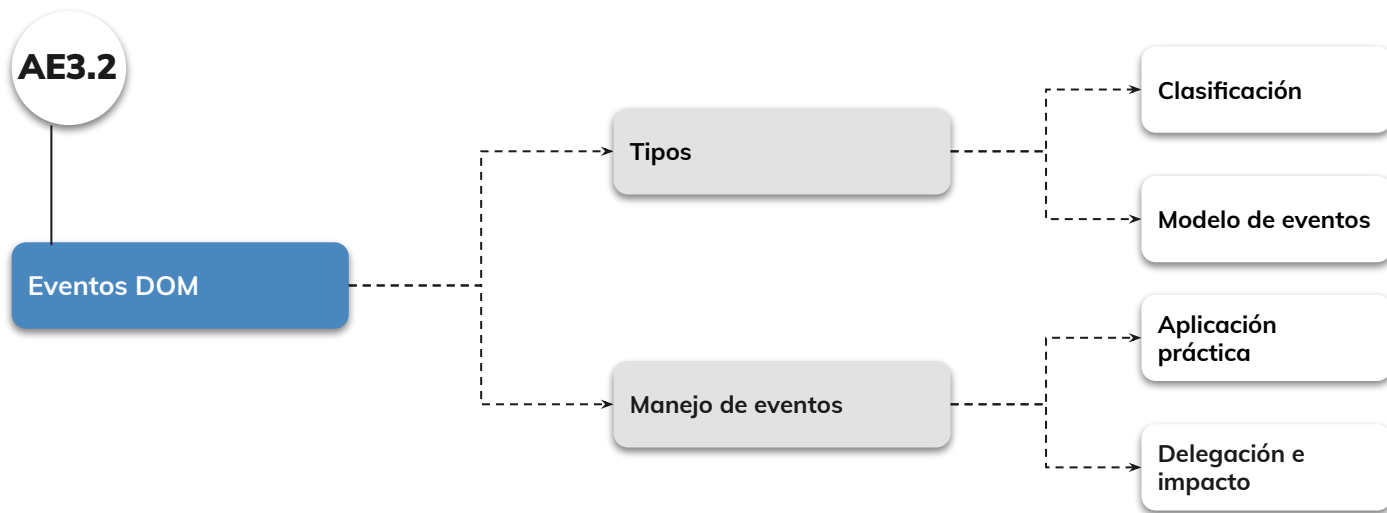
2

JavaScript asíncrono

4

Learning Path

¿Cuáles temas trabajaremos hoy?





Objetivos de aprendizaje

¿Qué aprenderás?



- Comprender qué son los eventos en JavaScript y cómo funcionan.
- Diferenciar los principales tipos de eventos y sus usos.
- Aplicar correctamente el modelo de eventos (event bubbling y event capturing).
- Implementar diferentes formas de manejar eventos en JavaScript.
- Utilizar la delegación de eventos para mejorar el rendimiento del código.

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Comprendimos el concepto de DOM y cómo JavaScript lo utiliza para interactuar con una página web.
- Exploramos la jerarquía del DOM, entendiendo la relación entre nodos y elementos.
- Aprendimos a seleccionar elementos del DOM con métodos como getElementById y querySelector.
- Modificamos elementos del DOM cambiando su contenido, estilos y atributos.
- Agregamos y eliminamos elementos dinámicamente, creando páginas más interactivas.

Tipos de eventos de DOM



¿Qué son los eventos en JavaScript?

Los eventos en JavaScript son acciones o sucesos que ocurren en el navegador y que pueden ser detectados y manejados mediante código. Permiten interactuar con los elementos del DOM y responder a acciones del usuario, como clics, movimientos del mouse o cambios en un campo de formulario.

```
document.getElementById("miBoton").addEventListener("click", function() {  
    alert("¡Botón clickeado!");  
});
```



Tipos de eventos en JavaScript

Clasificación de eventos más comunes:

- Eventos del Mouse: click, dblclick, mouseover, mouseout, mousemove
- Eventos del Teclado: keydown, keyup, keypress
- Eventos de Formulario: submit, change, focus, blur, input
- Eventos de la Ventana: load, resize, scroll, unload
- Eventos de Dispositivos Móviles: touchstart, touchmove, touchend
- Eventos del DOM: DOMContentLoaded, DOMNodeInserted, DOMNodeRemoved

Estos eventos permiten mejorar la experiencia del usuario en la web al reaccionar a diferentes interacciones.



Modelo de eventos en JavaScript



¿Cómo funcionan los eventos en JavaScript?

El modelo de eventos sigue dos enfoques principales:

1. Event Bubbling (Burbujeo de eventos):
 - El evento se propaga desde el elemento objetivo hacia sus elementos padres.
 - Ejemplo: Si hacemos clic en un botón dentro de un div, el evento se propaga desde el botón hasta el div.
2. Event Capturing (Captura de eventos):
 - El evento se propaga desde el elemento raíz del DOM hasta el elemento objetivo.



Modelo de eventos en JavaScript



```
const contenedor = document.getElementById("contenedor");
const boton = document.getElementById("boton");

contenedor.addEventListener("click", function() {
    console.log("Click en contenedor");
});

boton.addEventListener("click", function(event) {
    console.log("Click en botón");
    event.stopPropagation(); // Detiene la propagación del evento
});
```

En este caso, el `stopPropagation()` evita que el clic en el botón active también el evento del div.



Manejo de eventos en JavaScript



Existen varias formas de manejar eventos en JavaScript.

- Atributos HTML (No recomendado)

✓ Simple, pero difícil de mantener en código complejo.

```
<button onclick="alert('Hola')">Haz clic</button>
```

- Propiedades del DOM

✓ Más estructurado, pero solo permite un manejador por evento.

```
const btn = document.getElementById("miBoton");  
btn.onclick = function() {  
    alert("Botón clickeado");  
};
```



Manejo de eventos en JavaScript



- Event Listener (Recomendado)

✓ Permite múltiples funciones para un mismo evento y mejor control.

```
const btn = document.getElementById("miBoton");  
btn.addEventListener("click", function() {  
    alert("Botón clickeado");  
});
```



Delegación de eventos

Es una técnica eficiente donde un solo event listener en un elemento padre maneja los eventos de sus hijos, en lugar de asignar eventos a cada uno individualmente.

¿Por qué usarla?

- ✓ Reduce el uso de memoria y mejora el rendimiento.
- ✓ Facilita la manipulación de elementos dinámicos agregados al DOM.

En este caso, no es necesario agregar un click a cada LI, ya que el evento se maneja desde el UL.

```
document.getElementById("lista").addEventListener("click", function(event) {  
    if (event.target.tagName === "LI") {  
        alert("Clic en " + event.target.textContent);  
    }  
});
```

Eventos en JavaScript y su Aplicación Práctica

¿En qué consistirá la Demo?

En esta demo, aprenderemos cómo funcionan los eventos en JavaScript y cómo podemos aplicarlos para mejorar la interactividad de una página web. Implementaremos un gestor de tareas dinámico, donde los usuarios podrán agregar y eliminar tareas haciendo uso de:

- ✓ Manejo de eventos (addEventListener)
- ✓ Modelo de eventos (bubbling y capturing)
- ✓ Delegación de eventos
- ✓ Eventos del teclado y del mouse

Al finalizar la demo, los alumnos comprenderán cómo capturar eventos, cómo prevenir su propagación y cómo mejorar la eficiencia del código usando delegación de eventos.

Eventos en JavaScript y su Aplicación Práctica

1 Creación de la estructura HTML

📌 *Objetivo: Diseñar la estructura de la aplicación.*

♦ *Explicación:*

- *Se crea una interfaz con un campo de entrada (<input>), un botón (<button>) y una lista () para mostrar las tareas.*
- *Se enlaza un archivo script.js donde implementaremos la lógica.*

Eventos en JavaScript y su Aplicación Práctica

2 Agregar eventos a los elementos

 **Objetivo:** Capturar el evento click para agregar tareas.

◆ Explicación:

- Se captura el evento click en el botón "Agregar".
- Se valida que el campo de entrada no esté vacío.
- Se crea un nuevo elemento `<li>` y se agrega a la lista.

◆ Código JavaScript (script.js):

```
const tareaInput = document.getElementById("tareaInput");

const agregarTarea = document.getElementById("agregarTarea");

const listaTareas = document.getElementById("listaTareas");

agregarTarea.addEventListener("click", function() {

    if (tareaInput.value.trim() !== "") {

        const nuevaTarea = document.createElement("li");

        nuevaTarea.textContent = tareaInput.value;

        listaTareas.appendChild(nuevaTarea);

        tareaInput.value = "";

    }

});
```

Tiempo: 15 min

Acceder a elementos del DOM

3 Delegación de eventos

 **Objetivo:** Eliminar tareas usando delegación de eventos.

- ◆ Explicación:

- En lugar de agregar un event listener a cada `<li>`, lo colocamos en la `<ul>` (delegación de eventos).
- Cuando se hace clic en una tarea (`<li>`), esta se elimina.

- ◆ Código actualizado (script.js):

```
listaTareas.addEventListener("click",  
function(event) {  
    if (event.target.tagName === "LI") {  
        event.target.remove();  
    }  
});
```

Acceder a elementos del DOM

4 Manejo de eventos del teclado

📌 Objetivo: Agregar tareas presionando la tecla "Enter".

♦ Explicación:

- Se escucha el evento `keypress` en el campo de entrada.
- Si el usuario presiona "Enter", se activa la función del botón "Agregar".

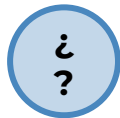
♦ Código actualizado (script.js):

```
tareaInput.addEventListener("keypress",  
function(event) {  
  if (event.key === "Enter") {  
    agregarTarea.click(); // Simula el clic en el botón  
  }  
});
```

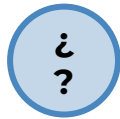
#Momentode Preguntas...



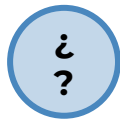
¿Cómo interactúa el usuario con una página web a través de eventos?



¿Por qué es importante entender el modelo de burbujeo y captura de eventos?



¿Cuáles son las ventajas y desventajas de asignar eventos directamente en HTML, usando propiedades del DOM o con `addEventListener`?



¿Cómo podríamos mejorar el rendimiento de una página web al manejar eventos en elementos dinámicos?



Momento:

Time-out!



5 -10 min.





Ejercicio N° 01

Agregar un elemento al DOM

Agregar un elemento al DOM

Contexto: 🙌

Podemos agregar elementos nuevos al DOM con `createElement` y `appendChild`, y podemos manejar eventos de diversas maneras en JavaScript.

Consigna: 🛠️

Crea un botón que, al hacer clic, agregue un nuevo párrafo al div.

Paso a paso: ⚙️

Implementa el manejo de eventos de todas las formas posibles:

1. Mediante el atributo `onclick` en HTML.
2. Usando la propiedad `onclick` en JavaScript.
3. Con `addEventListener`.

Tiempo 🕒: 15 min

Agregar un elemento al DOM

```
<button id="btnAgregar">Agregar Párrafo</button>
<div id="contenedor"></div>
```

```
<script>
  function agregarParrafo() {
    const nuevoP = document.createElement("p");
    nuevoP.textContent = "Este es un nuevo párrafo";
    document.getElementById("contenedor").appendChild(nuevoP);
  }
```

// Método 1: Atributo en HTML (onclick="agregarParrafo()")

// Método 2: Asignando el evento con propiedad onclick
document.getElementById("btnAgregar").onclick = agregarParrafo;

// Método 3: Usando addEventListener
document.getElementById("btnAgregar").addEventListener("click", agregarParrafo);
</script>



Ejercicio N° 02

Modificar un estilo del DOM

Modificar un estilo del DOM

Contexto: 🙌

Podemos modificar los estilos de un elemento cambiando style o usando classList, y podemos manejar eventos de diversas formas.

Consigna: 🛠️

Al hacer clic en un botón, cambia el color de un texto a azul.

Tiempo 🕒: 20 min

Paso a paso: ⚙️

Alterna la clase "resaltado" en otro párrafo. Implementa el manejo de eventos de todas las formas posibles:

1. Mediante el atributo onclick en HTML.
2. Usando la propiedad onclick en JavaScript.
3. Con addEventListener.

Modificar un estilo del DOM

```
<p id="texto1">Este texto cambiará de color.</p>
<p id="texto2">Este texto cambiará de clase.</p>
<button id="btnEstilos">Modificar Estilos</button>
```

```
<style>
  .resaltado {
    color: red;
    font-weight: bold;
  }
</style>
```

```
<script>
  function cambiarEstilos() {
    document.getElementById("texto1").style.color = "blue";
    document.getElementById("texto2").classList.toggle("resaltado");
  }
</script>
```

```
// Método 2: Asignando el evento con propiedad onclick
document.getElementById("btnEstilos").onclick = cambiarEstilos;
```

```
// Método 3: Usando addEventListener
document.getElementById("btnEstilos").addEventListener("click", cambiarEstilos);
</script>
```



Ejercicio N° 03

Delegación de eventos

Delegación de eventos

Contexto: 🙌

Cuando agregamos elementos dinámicamente al DOM, no podemos asignar eventos directamente a ellos si no existen al cargar la página. Para esto, usamos la delegación de eventos.

Consigna: 📝

Crea una lista en la que cada elemento tenga un botón para eliminarse. Usa delegación de eventos para que al hacer clic en cualquier botón, se elimine el elemento correspondiente.

Paso a paso: ⚙️

Implementa el manejo de eventos de todas las formas posibles:

1. Usando el atributo onclick en HTML.
2. Asignando el evento con la propiedad onclick en JavaScript.
3. Utilizando addEventListener.
4. Aplicando delegación de eventos para manejar clics en los botones de los elementos generados dinámicamente.

Tiempo 🕒: 20 min

Delegación de eventos

```
<ul id="lista">
  <li>Elemento 1 <button class="eliminar">Eliminar</button></li>
  <li>Elemento 2 <button class="eliminar">Eliminar</button></li>
  <li>Elemento 3 <button class="eliminar">Eliminar</button></li>
</ul>

<script>
  document.getElementById("lista").addEventListener("click", function(event) {
    if (event.target.classList.contains("eliminar")) {
      event.target.parentElement.remove();
    }
  });
</script>
```



Ejercicio N° 04

Capturando eventos del teclado

Capturando eventos del teclado

Contexto: 🙌

Podemos capturar eventos de teclado como keydown, keyup y keypress. Además, podemos prevenir el comportamiento predeterminado de algunos eventos.

Consigna: 📝

Crea un campo de texto donde los usuarios puedan escribir, pero evita que se ingresen números. Muestra en tiempo real el contenido en un div debajo del campo.

Paso a paso: ⚙️

Implementa el manejo de eventos de todas las formas posibles:

1. Usando el atributo oninput en HTML.
2. Asignando el evento con la propiedad oninput en JavaScript.
3. Utilizando addEventListener para capturar eventos de entrada.
4. Usando keydown para evitar la escritura de números en tiempo real.

Tiempo 🕒: 20 min

Capturando eventos del teclado

```
<input type="text" id="inputTexto" placeholder="Escribe aquí">
<div id="resultado"></div>

<script>
  document.getElementById("inputTexto").addEventListener("input", function(event) {
    document.getElementById("resultado").textContent = event.target.value;
  });

  document.getElementById("inputTexto").addEventListener("keydown", function(event) {
    if (!isNaN(event.key) && event.key !== " ") {
      event.preventDefault();
    }
  });
</script>
```

Eventos avanzados y propagación

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Eventos Avanzados en JavaScript</title>
</head>
<body>
  <div id="contenedor" style="padding: 20px; border: 2px solid black;">
    <p id="parrafo">Pasa el mouse por aquí o haz doble clic.</p>
  </div>
  <button id="botonDerecho">Haz clic derecho aquí</button>
  <script>
// Evento Mouseover
document.getElementById("parrafo").addEventListener("mouseover",
function() {
  this.style.color = "blue";
  console.log("Mouse sobre el párrafo");
});
// Evento Doble Clic
document.getElementById("parrafo").addEventListener("dblclick", function() {
  this.style.fontSize = "20px";
  alert("Doble clic en el párrafo");
});
```

```
// Evento Contextmenu (Clic derecho)
document.getElementById("botonDerecho").addEventListener("contextmenu", function(event) {
  event.preventDefault();
  alert("Clic derecho deshabilitado en este botón");
});
// Control de Propagación de eventos
document.getElementById("contenedor").addEventListener("click", function() {
  alert("Evento en el contenedor");
});
document.getElementById("parrafo").addEventListener("click", function(event) {
  event.stopPropagation(); // Evita que el clic llegue al contenedor
  alert("Evento en el párrafo");
});
</script>
</body>
</html>
```

Desafío: Identificar el error

```
<button id="miBoton">Clic</button>

<script>
  document.getElementById("miBoton").addEventListener("click", function() {
    alert("Botón presionado");
  });
</script>
```

Pregunta: ¿Qué está mal en el siguiente código y cómo solucionarlo?

Opciones:

1. addEventListener está mal escrito, debería ser addEventListener
2. El botón no tiene un ID válido.
3. alert() no funciona en eventos de clic.
4. Falta cerrar la etiqueta button.

Desafío: ¿Qué hace este código?

```
<input type="text" id="entrada">

<script>
  document.getElementById("entrada").addEventListener("keydown", function(event) {
    console.log(event.key);
  });
</script>
```

Pregunta: ¿Cuál es la salida del siguiente código cuando presionamos la tecla "a"?

Opciones:

1. Nada, el código tiene errores.
2. Muestra en la consola "a" si presionamos la tecla "a".
3. Cambia el contenido del input automáticamente.
4. Borra el texto ingresado.

Desafío: ¿Cuál es el resultado esperado?

```
<button id="miBoton">Haz clic</button>

<script>
  document.getElementById("miBoton").addEventListener("click", function() {
    console.log("Clic detectado");
  });

  document.getElementById("miBoton").addEventListener("click", function() {
    console.log("Otro evento de clic");
  });
</script>
```

Pregunta: ¿Cuál es la salida del siguiente código cuando presionamos la tecla "a"?

Opciones:

1. Nada, el código tiene errores.
2. Muestra en la consola "a" si presionamos la tecla "a".
3. Cambia el contenido del input automáticamente.
4. Borra el texto ingresado.

Desafío: ¿Cómo evitar que el formulario se envíe?

```
<form id="miFormulario">
  <button type="submit">Enviar</button>
</form>

<script>
  document.getElementById("miFormulario").addEventListener("submit", function(event) {
    // Completa el código
  });
</script>
```

Pregunta: ¿Cuál es la forma correcta de evitar el envío de un formulario al hacer clic en el botón de enviar?

Opciones:

1. event.preventDefault();
2. event.stopPropagation();
3. this.reset();
4. console.log("Formulario enviado");

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ Entendimos qué son los eventos en JavaScript y cómo se clasifican.
- ✓ Aprendimos sobre event bubbling y event capturing y cómo controlarlos.
- ✓ Implementamos distintas formas de manejar eventos de manera eficiente.
- ✓ Exploramos la delegación de eventos y su impacto en el rendimiento.



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compártela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

